

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении практической работы №2
по дисциплине «Технологии разработки программного
обеспечения»
«Разработка клиент-серверного приложения»

Студент
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Силиненко А.В.

«_____» _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Спецификация	4
1.1 Общее описание задачи	4
1.2 Описание интерфейса	4
1.3 Функциональные требования	6
1.4 Требования к тестированию	9
2 Проектирование	10
2.1 Перечень модулей	10
2.1.1 Клиентская часть	10
2.1.2 Серверная часть	10
2.2 Алгоритмы работы клиентской и серверной частей приложения	10
2.3 Форматы структур данных	13
3 Разработка программы	14
4 Тестирование	16
Заключение	17
Список использованных источников	18
Приложение 1. Исходный код клиента	19
Приложение 2. Исходный код сервера	25
Приложение 3. Исходный код тестирующего приложения	29
Приложение 4. Протоколы тестирования	43

Введение

Данная работа посвящена разработке клиент-серверного приложения, вычисляющего длину минимального пути в графе для операционной системы Linux.

Актуальность работы заключается в приобретении практического опыта в разработке на C++, изучения особенностей создания программного обеспечения в операционной системе Linux и понимания реализации полного цикла создания программного продукта.

Цель работы: разработка клиент-серверного приложения, вычисляющего длину минимального пути в неориентированном графе.

Задачи:

1. Получение знаний об этапах разработки программного обеспечения.
2. Получение базовых знаний по разработке приложений в ОС Linux.
3. Получение знаний и практических навыков реализации межпроцессных взаимодействий с помощью механизма сокетов.
4. Получение знаний по работе с графами.
5. Разработка клиент-серверного приложения.
6. Тестирование клиент-серверного приложения.

1 Спецификация

1.1 Общее описание задачи

Назначение программы: вычисление длины минимального пути в неориентированном графе.

Архитектура: x86.

Операционная система: Linux

Язык реализации: C/C++.

1.2 Описание интерфейса

1.2.1. Входные данные

2.1.1. Входные данные сервера

2.1.1.1. При запуске тип подключения <TCP|UDP>

2.1.1.2. Пакет, отправленный через сокет клиентом и включающий: список ребер, число ребер, начальная и конечная вершина

2.1.2 Входные данные клиента

2.1.2.1. При запуске тип подключения <TCP|UDP>, IP-адрес сервера и порт серверной части

2.1.2.2. Строка с ответом от сервера, полученная через сокет

2.1.2.3. Данные, вводимые пользователем с клавиатуры или из файла

1.2.2. Выходные данные

2.2.1. Для сервера TCP/UDP

2.2.1.1. Строка с вычисленным значением минимального пути графа и списком ребер минимального пути, переданная через сокет

2.2.1.2. Вывод на экран подтверждения об успешной отправке данных

2.2.2. Для клиента TCP/UDP

2.2.2.1. Вывод строки с вычисленным значением минимального пути графа и списком ребер минимального пути

2.2.2.2. Вывод на экран подтверждения об успешной отправке и получении данных

2.2.3.1. Сообщения серверной части

- 2.2.3.1.1. Информационное «Сервер запущен по протоколу TCP/UDP, порт <port>»
- 2.2.3.1.2. Информационное «Новое подключение!»
- 2.2.3.1.3. Информационное «Клиент отключился»
- 2.2.3.1.4. Информационное «Пакет получен, отправляю АСК» — при реализации надёжной доставки по UDP
- 2.2.3.1.5. Информационное «Отправка ответа клиенту»
- 2.2.3.1.6. Ошибка «Потеряна связь с клиентом» — при отсутствии подтверждения после 3 повторных отправок
- 2.2.3.1.7. Ошибка «Ошибка создания сокета» (socket() failed)
- 2.2.3.1.8. Ошибка «Ошибка привязки сокета» (bind() failed)
- 2.2.3.1.9. Ошибка «Ошибка при прослушивании соединений» (listen() failed)
- 2.2.3.1.10. Ошибка «Ошибка принятия соединения» (accept() failed)
- 2.2.3.1.11. Ошибка «Ошибка получения данных от клиента» (recv() failed)
- 2.2.3.1.12. Ошибка «Ошибка отправки сообщения клиенту» (send() failed)
- 2.2.3.1.13. Ошибка «Граф не является связным!»
- 2.2.3.1.14. Ошибка «Количество рёбер должно быть не менее 6»

2.2.3.2. Сообщения клиентской части

- 2.2.3.2.1. Информационное «Подключение к TCP/UDP серверу»
- 2.2.3.2.2. Информационное «Отключение от сервера...»
- 2.2.3.2.3. Информационное «Запуск программы "Сетевое взаимодействие TCP/UDP: поиск минимального пути в графе»>
- 2.2.3.2.4. Информационное сообщение «Вывод справочной информации (-help): поддерживаемые команды и параметры запуска программы»
- 2.2.3.2.5. Информационное «Отправлено подтверждение (АСК)» — при реализации надёжной доставки по UDP
- 2.2.3.2.6. Информационное «Получен АСК от сервера» — при реализации надёжной доставки по UDP
- 2.2.3.2.7. Приглашение «Введите начальную и конечную вершины»

- 2.2.3.2.8. Приглашение «Введите рёбра в формате "источник цель вес» >
 - 2.2.3.2.9. Подсказка «Для выхода введите exit»
 - 2.2.3.2.10. Ошибка «Неверный формат ввода данных»
 - 2.2.3.2.11. Ошибка «Неверное количество аргументов командной строки»
 - 2.2.3.2.12. Ошибка «Неизвестный протокол. Используйте TCP или UDP»
 - 2.2.3.2.13. Ошибка «Неверный формат IP-адреса»
 - 2.2.3.2.14. Ошибка «Номер порта должен быть в диапазоне 1–65535»
 - 2.2.3.2.15. Ошибка «Потеряна связь с сервером»
- 1.2.3. Хранимые данные одинаковы для клиента и сервера: структура данных для представления графа, представляющая собой количество вершин и ребер.

1.3 Функциональные требования

3.1. Ввод данных

- 3.1.1. Серверная часть должна выводить сообщение о запуске сервера в соответствии с п. 2.2.3.1.1.
- 3.1.2. Серверная часть должна выводить сообщение о подключении клиента в соответствии с п. 2.2.3.2.1.
- 3.1.3. Серверная часть должна выводить сообщение о получении запроса от клиента в соответствии с п. 2.2.3.1.2.
- 3.1.4. Клиентская часть должна выводить приглашения для ввода данных о графе в соответствии с пп. 2.2.3.2.7 – 2.2.3.2.8, а также подсказку в соответствии с п. 2.2.3.2.9.
- 3.1.5. Клиентская часть должна вводить с клавиатуры структуру данных в соответствии с п. 2.1.2.3 (вершины + рёбра + веса + начальная и конечная вершины).
- 3.1.6. Клиентская часть должна предоставлять возможность выхода по команде exit в соответствии с п. 2.2.3.2.9.
- 3.1.7. При запуске клиентской части должна выводиться информационная шапка программы в соответствии с п. 2.2.3.2.3.
- 3.1.8. При запуске программы с ключом –help должна выводиться справочная информация в соответствии с п. 2.2.3.2.4.

3.2. Проверка корректности данных

- 3.2.1. Серверная часть должна проверить успешность создания сокета; при ошибке выводится сообщение в соответствии с п. 2.2.3.1.7.
- 3.2.2. Серверная часть должна проверить успешность привязки сокета сервера к адресу; при ошибке выводится сообщение в соответствии с п. 2.2.3.1.8.
- 3.2.3. Серверная часть должна проверить успешность перевода сокета в режим прослушивания; при ошибке выводится сообщение в соответствии с п. 2.2.3.1.9.
- 3.2.4. Серверная часть должна проверить успешность принятия входящего соединения; при ошибке выводится сообщение в соответствии с п. 2.2.3.1.10.
- 3.2.5. Серверная часть должна проверить успешность установки соединения с клиентом; при ошибке выводится сообщение в соответствии с п. 2.2.3.2.15.
- 3.2.6. Клиентская часть должна проверить успешность отправки данных от клиента серверу; при ошибке выводится сообщение в соответствии с п. 2.2.3.1.12.
- 3.2.7. Серверная часть должна проверить успешность получения данных от клиента сервером; при ошибке выводится сообщение в соответствии с п. 2.2.3.1.11.
- 3.2.8. Серверная часть должна проверить успешность отправки данных от сервера клиенту; при ошибке выводится сообщение в соответствии с п. 2.2.3.1.12.
- 3.2.9. Клиентская часть должна проверить успешность получения данных от сервера клиентом; при ошибке выводится сообщение в соответствии с п. 2.2.3.2.15.
- 3.2.10. Клиентская часть должна проверить корректность формата вводимых данных; при ошибке выводится сообщение в соответствии с п. 2.2.3.2.10.
- 3.2.11. Программа должна проверить корректность параметров командной строки при запуске сервера или клиента; при ошибке выводится сообщение в соответствии с п. 2.2.3.2.12 – 2.2.3.2.14.
- 3.2.12. Серверная часть должна проверить связность графа; при нарушении выводится сообщение в соответствии с п. 2.2.3.1.13.

3.2.13. Клиентская часть должна проверить корректность количества вершин графа (не менее 6 и не более 100); при нарушении выводится сообщение в соответствии с п. 2.2.3.1.14.

3.3. Вычисление минимального пути в графе

3.3.1. Серверная часть должна реализовать алгоритм поиска минимального пути в графе с помощью алгоритма Дейкстры

3.3.2. Серверная часть должна корректно работать с неотрицательными весами рёбер.

3.3.3. Серверная часть должна поддерживать одновременную работу с минимум 3 клиентами.

3.3.4. Серверная часть должна поддерживать оба протокола передачи данных (TCP и UDP).

3.3.5. При работе по протоколу UDP должна быть реализована надёжная доставка сообщений, включая:

3.3.5.1. Отправку подтверждения приёма (ACK) — в соответствии с п. 2.2.3.1.4 (сервер) и 2.2.3.2.5 (клиент);

3.3.5.2. Получение подтверждения (ACK) — в соответствии с п. 2.2.3.1.5 (сервер) и 2.2.3.2.6 (клиент);

3.3.5.3. Повторную отправку сообщения при отсутствии подтверждения в течение 3 секунд;

3.3.5.4. При трёх неудачных попытках клиент должен вывести сообщение в соответствии с п. 2.2.3.2.15, а сервер - с п. 2.2.3.1.6.

3.3.6. Серверная часть должна проверить связность графа, если условие не выполняется, на экран выводится ошибка в соответствии с п. 2.2.3.1.13

3.4. Вывод результатов

3.4.1. Клиентская часть должна выводить строку «Минимальный путь = », вычисленное значение длины минимального пути и список вершин этого пути

3.4.2. При получении клиентской частью строки «exit», на экран клиентской части должно вывестись информационное сообщение в соответствии с п. 2.2.3.2.2, и программа должна корректно завершить соединение

- 3.4.3. В момент завершения работы клиентской части по протоколу ТСР, на экран серверной части должно быть выведено информационное сообщение в соответствии с п. 2.2.3.1.3
- 3.4.4. При успешной передаче подтверждения (АСК) клиент и сервер должны выводить соответствующие сообщения в соответствии с пп. 2.2.3.1.5 и 2.2.3.2.6
- 3.4.5. Серверная часть должна корректно завершаться по получению команды «kill»

1.4 Требования к тестированию

Необходимо разработать автоматические тесты, обеспечивающие тестирование разработанного приложения в следующем объеме:

- 1.4.1. Работа приложения по протоколу ТСР.
- 1.4.2. Работа приложения по протоколу UDP, включая работу клиента при недоступности сервера.
- 1.4.3. Ввод исходных данных с клавиатуры.
- 1.4.4. Ввод исходных данных из файла.
- 1.4.5. Работа приложения при задании графа, количество вершин которого близко к граничным значениям области допустимых значений (ОДЗ), заданном в п.3.2.13.:
 - граф с 5 вершинами (на единицу меньше нижней границы ОДЗ);
 - граф с 6 вершинами (равно нижней границе ОДЗ);
 - граф с 100 вершинами (равно верхней границе ОДЗ);
 - граф с 101 вершиной (на единицу больше верхней границы ОДЗ);
- 1.4.6. Работа приложения при задании значений количества вершин из середины ОДЗ (10 вершин).

2 Проектирование

2.1 Перечень модулей

2.1.1 Клиентская часть

1. Модуль ввода данных используется для получения и валидации входных данных. Основные функции - парсинг описания графа, проверка корректности данных и преобразование в внутренний формат.
2. Модуль сетевого взаимодействия используется для управления сетевым соединением с сервером. Основные функции - установка и разрыв соединения, отправка данных на сервер и прием ответов от него, обработка таймаутов и повторных отправок (для UDP).
3. Модуль пользовательского интерфейса для взаимодействия с пользователем. Основные функции - отображение меню, обработка команды выхода и вывод результатов расчета.
4. Модуль настроек приложения. Основные функции - парсинг аргументов командной строки и управление параметрами подключения.

2.1.2 Серверная часть

1. Модуль сетевого взаимодействия используется для управления несколькими клиентскими соединениями. Основные функции - прослушивание портов, прием запросов от клиентов и отправка им ответов, мультиплексирование соединений.
2. Модуль обработки графа используется для вычисления минимальных путей. Основные функции - валидация структуры графа, построение графа из входных данных, реализация алгоритма Дейкстры.
3. Модуль управления настройками приложения. Основные функции - управление портами и адресами, чтение параметров запуска.

2.2 Алгоритмы работы клиентской и серверной частей приложения

На рисунках 1- 2 представлены схемы алгоритмов работы клиентской и серверной частей приложения при функционировании по прото-

колу UDP.

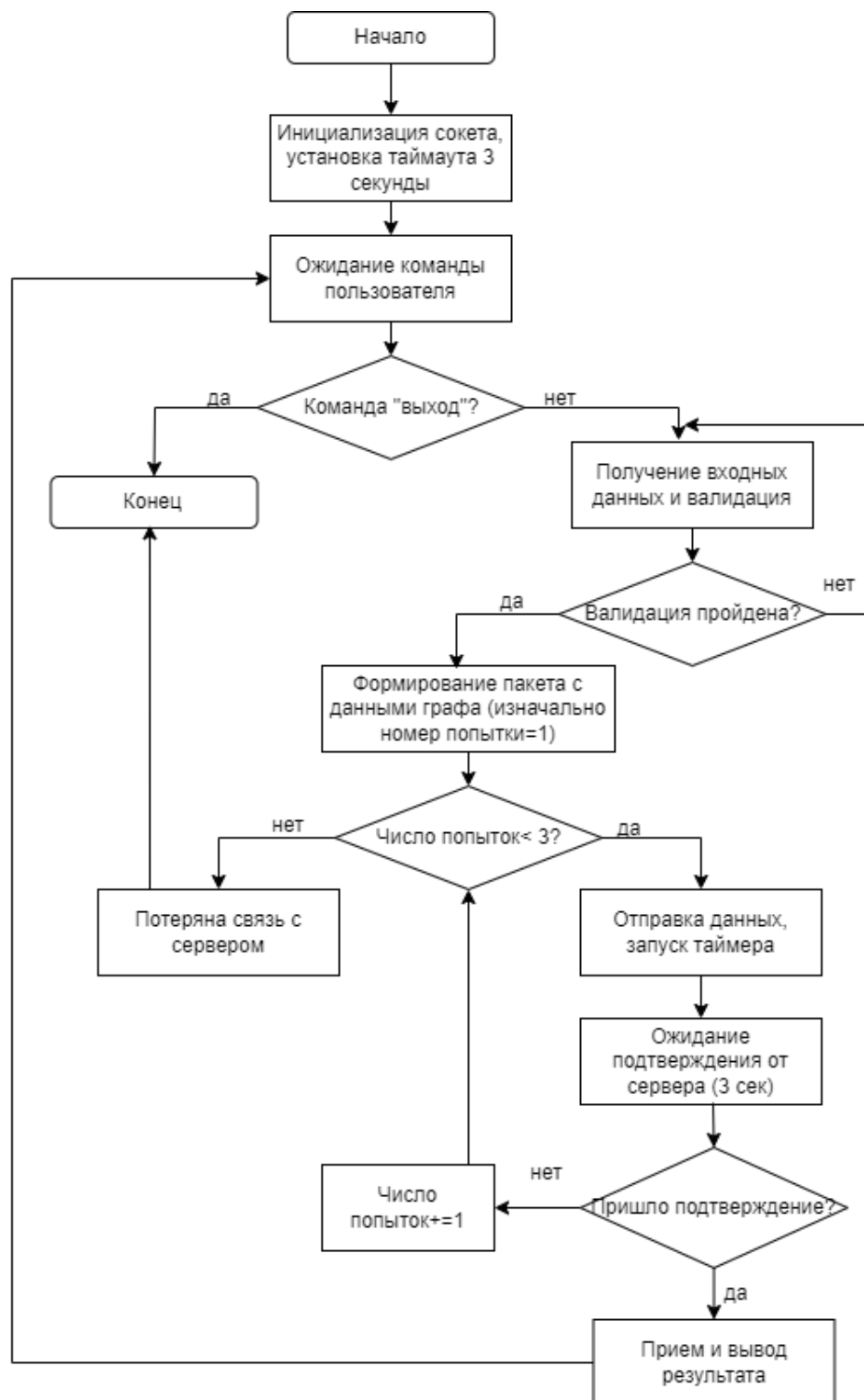


Рис. 1. Алгоритм работы клиентской части

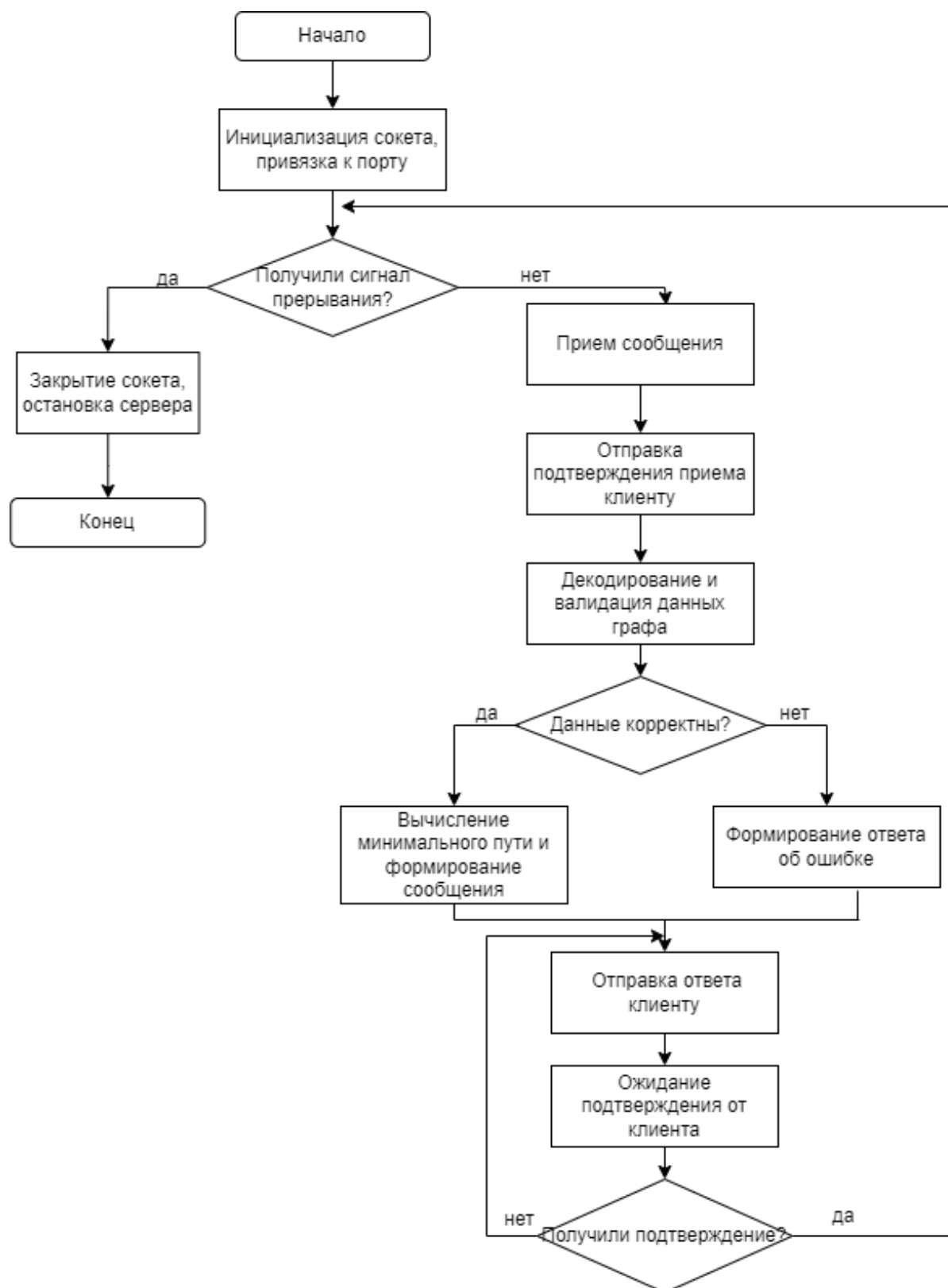


Рис. 2. Алгоритм работы серверной части

2.3 Форматы структур данных

Ребро Edge представлена исходной вершиной u , конечной v и весовой вершиной w .

Листинг 1. Структура ребра

```
1 typedef struct {  
2     int u, v, w;  
3 } Edge;
```

ClientInfo используется для обслуживания одного клиента. Каждый поток получает такую информацию, чтобы иметь номер сокета, тип соединения (TCP/UDP), адрес клиента.

Листинг 2. Структура информации для потока

```
1 typedef struct {  
2     int sock;  
3     struct sockaddr_in addr;  
4     int is_tcp;  
5 } ClientInfo;
```

Для передачи данных от клиента серверу используется массив целых чисел, где в первых трех элементах записаны данные о количестве ребер (m), начальной (s) и конечной (t) вершин пути. Далее передаются m троек ребер структуры Edge.

Листинг 3. Запрос от клиента серверу

```
1 int buf[3 + m * 3];
```

Для передачи данных от сервера клиенту используется строка с результатом вида «Длина: $\%d$; Путь: ...».

Листинг 4. Ответ от сервера клиенту

```
1 char response[1000];
```

Размер буфера выбран равным 1000 байт, чтобы обеспечить корректную работу для всего объема допустимых значений графа, в том числе верхней границы (100 вершин, соответственно, максимальный путь состоит из 99 ребер). При максимальной длине пути и больших значениях весов общий размер передаваемой строки - описания кратчайшего пути не превышает response. Данный формат обеспечивает оптимальный объем передаваемых через сеть данных, с возможностью увеличения максимального количества вершин.

3 Разработка программы

Основные сведения о процессе разработки и разработанном приложении:

3.1. Среда разработки - Visual Studio Code

3.2. Компилятор - gcc

3.3. Примеры компиляции и запуска приложения приведены в листинге 3. Исходный код клиент-серверного приложения приведен в приложениях 1 (полный код клиента client.c) и 2 (полный код сервера server.c)

Листинг 5. Примеры запуска приложения на компиляцию и сборку

```
1 gcc -o server server.c
2 gcc -o client client.c
3
4 ./server tcp 8080
5 ./client 127.0.0.1 tcp 8080
```

3.4. Основные технологии, используемые в приложении:

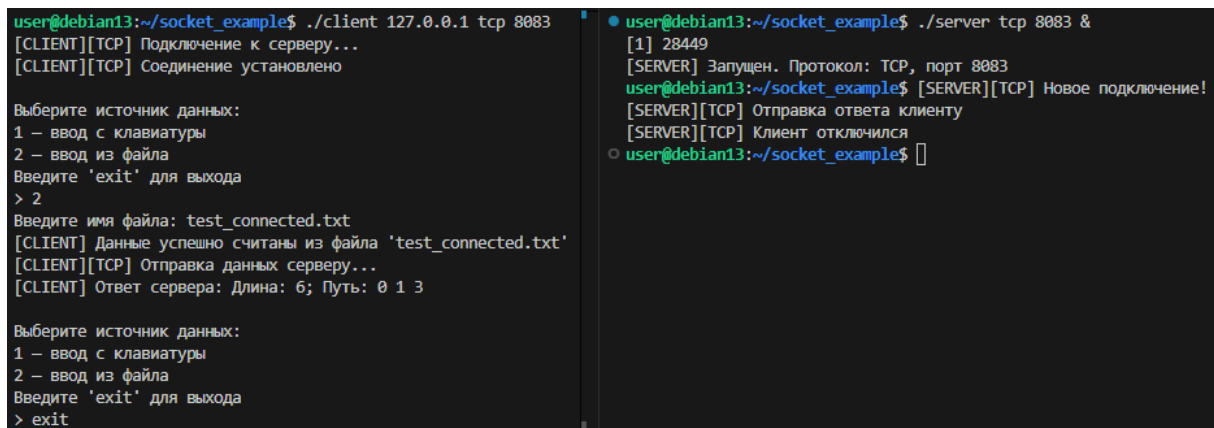
3.4.1. Механизм межпроцессного взаимодействия:

- `socket(AF_INET, SOCK_STREAM/ SOCK_DGRAM, 0)` - создание TCP/UDP сокета.
- `bind()` - привязка сокета к адресу и порту на сервере.
- `listen()/accept()` - перевод сокета в режим прослушивания и принятие входящих соединений (TCP).
- `connect()` - установка соединения на стороне клиента (TCP).
- `send()/recv()` - отправка/приём данных при TCP.
- `sendto()/recvfrom()` - отправка/приём при UDP.
- `select()` - используется на клиенте для ожидания ACK с таймаутом 3 секунды при реализации надёжной доставки UDP.
- `pthread_create()` / `pthread_detach()` - создание потоков для обработки каждого TCP-клиента на сервере.
- `htons()` - преобразование адресов и портов.
- `perror()` и `exit()` - обработка критических ошибок.

3.4.2. Механизм мультиплексирования ввода/вывода:

TCP: основной поток вызывает ассепт, для каждого ассепт создается pthread, который выполняет client_thread, что обеспечивает параллельную обработку нескольких TCP-клиентов.

3.3.3. На рисунках 3 - 4 представлены примеры запуска и работы клиента и сервера



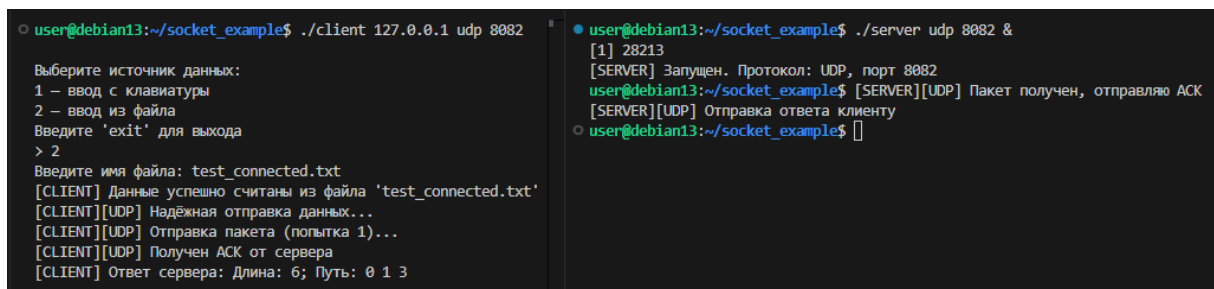
```
user@debian13:~/socket_example$ ./client 127.0.0.1 tcp 8083
[CLIENT][TCP] Подключение к серверу...
[CLIENT][TCP] Соединение установлено

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 2
Введите имя файла: test_connected.txt
[CLIENT] Данные успешно считаны из файла 'test_connected.txt'
[CLIENT][TCP] Отправка данных серверу...
[CLIENT] Ответ сервера: Длина: 6; Путь: 0 1 3

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> exit

user@debian13:~/socket_example$ ./server tcp 8083 &
[1] 28449
[SERVER] Запущен. Протокол: TCP, порт 8083
user@debian13:~/socket_example$ [SERVER][TCP] Новое подключение!
[SERVER][TCP] Отправка ответа клиенту
[SERVER][TCP] Клиент отключился
user@debian13:~/socket_example$
```

Рис. 3. Работа клиента и сервера по протоколу TCP



```
user@debian13:~/socket_example$ ./client 127.0.0.1 udp 8082
Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 2
Введите имя файла: test_connected.txt
[CLIENT] Данные успешно считаны из файла 'test_connected.txt'
[CLIENT][UDP] Надёжная отправка данных...
[CLIENT][UDP] Отправка пакета (попытка 1)...
[CLIENT][UDP] Получен ACK от сервера
[CLIENT] Ответ сервера: Длина: 6; Путь: 0 1 3

user@debian13:~/socket_example$ ./server udp 8082 &
[1] 28213
[SERVER] Запущен. Протокол: UDP, порт 8082
user@debian13:~/socket_example$ [SERVER][UDP] Пакет получен, отправляю ACK
[SERVER][UDP] Отправка ответа клиенту
user@debian13:~/socket_example$
```

Рис. 4. Работа клиента и сервера по протоколу UDP

4 Тестирование

Тестирующее приложение было разработано с помощью скриптового языка Tcl и расширения expect.

Tcl (Tool Command Language) - это скриптовый язык программирования, созданный для управления приложениями. Основные команды:

- `open`, `puts`, `close` - генерация тестовых файлов.
- `fuser` - управление портами.
- `exec` - выполнение внешней программы и возврата вывода.
- `set` - хранение PID, путей, сообщений.

Expect позволяет создавать программы, ожидающие вопросов от других программ и дающие им ответы. Основные команды:

- `spawn` - запуск процесса или программы.
- `expect` - ожидание данных, выводимых программой. При написании скрипта можно указать, какого именно вывода он ждёт и как на него нужно реагировать.
- `send` - отправка ответа. Expect-скрипт с помощью этой команды может отправлять входные данные автоматизируемой программе.
- `interact` - позволяет переключиться на «ручной» режим управления программой.

Исходный полный код автотестов приведен в приложении 3. Протоколы тестирования в виде скриншотов с результатами прохождения тестов приведены в приложении 4.

Заключение

В ходе выполнения работы было разработано клиент-серверное приложение на C++, которое вычисляет длину минимального пути в неориентированном графе. Сообщение между клиентами и сервером может осуществляться по протоколам TCP и UDP, ввод данных возможен с клавиатуры или из файла. Для проверки корректности работы приложение было протестировано автоматическими тестами с помощью расширения exptest. Таким образом, все задачи были выполнены и цель работы достигнута.

Список литературы

- [1] Хабр : Bash-скрипты, часть 11: expect и автоматизация интерактивных утилит
URL: <https://habr.com/ru/companies/ruvds/articles/328436/>
(дата обращения 08.12.2025)
- [2] OpenNet : Основные понятия и элементы языка Tcl <https://www.opennet.ru/docs/RUS/tcltk/base.html> (дата обращения 08.12.2025)
- [3] Хабр : TCP и UDP <https://habr.com/ru/articles/711578/> (дата обращения 10.11.2025)

Приложение 1. Исходный код клиента

Листинг 6. Исходный код клиента

```
1 // client.c - клиентская часть: TCP/UDP, надёжная доставка UDP, ввод графа,
  вывод служебных сообщений
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <arpa/inet.h>
7 #include <sys/socket.h>
8 #include <fcntl.h>
9 #include <errno.h>
10 #include <time.h>
11 #include <sys/time.h>
12
13
14 #define MAX_EDGES 128
15 #define BUF_SIZE 2048
16
17 typedef struct {
18     int u, v;
19     int w;
20 } Edge;
21 // Аварийное завершение с выводом ошибки
22 void die(const char *msg) {
23     perror(msg);
24     exit(EXIT_FAILURE);
25 }
26
27
28 // Определение количества вершин в графе
29 int count_vertices(Edge *edges, int m) {
30     int max_vertex = 0;
31     int vertices_set[256] = {0}; // Массив для отслеживания уникальных вершин
32     int vertex_count = 0;
33
34     // Находим максимальный номер вершины
35     for (int i = 0; i < m; i++) {
36         if (edges[i].u > max_vertex) max_vertex = edges[i].u;
37         if (edges[i].v > max_vertex) max_vertex = edges[i].v;
38
39         // Добавляем вершины в set для подсчета уникальных
40         if (!vertices_set[edges[i].u]) {
41             vertices_set[edges[i].u] = 1;
42             vertex_count++;
43         }
44         if (!vertices_set[edges[i].v]) {
45             vertices_set[edges[i].v] = 1;
46             vertex_count++;
47         }
48     }
49     return vertex_count;
50 }
51
```

```

52 // Проверка количества рёбер
53 int check_edge_count(int m, int vertex_count) {
54     if (m < 6 || m > 100) {
55         return 0;
56     }
57
58     // Проверяем, не превышает ли количество рёбер максимально возможное
59     int max_possible_edges = (vertex_count * (vertex_count - 1)) / 2;
60     return m <= max_possible_edges;
61 }
62
63 // BFS для проверки связности
64 int check_connected(Edge *edges, int m, int vertex_count) {
65     if (m == 0 || vertex_count == 0) return 0;
66
67     // Находим максимальный номер вершины
68     int maxv = 0;
69     for (int i = 0; i < m; i++) {
70         if (edges[i].u > maxv) maxv = edges[i].u;
71         if (edges[i].v > maxv) maxv = edges[i].v;
72     }
73
74     // Создаем матрицу смежности
75     int g[maxv + 1][maxv + 1];
76     memset(g, 0, sizeof(g));
77
78     for (int i = 0; i < m; i++) {
79         g[edges[i].u][edges[i].v] = 1;
80         g[edges[i].v][edges[i].u] = 1;
81     }
82
83     // Находим первую вершину для BFS
84     int start_vertex = edges[0].u;
85     int visited[maxv + 1];
86     memset(visited, 0, sizeof(visited));
87
88     // BFS queue
89     int q[maxv + 1], qs = 0, qe = 0;
90     visited[start_vertex] = 1;
91     q[qe++] = start_vertex;
92     int visited_count = 1;
93
94     while (qs < qe) {
95         int v = q[qs++];
96         for (int u = 0; u <= maxv; u++) {
97             if (g[v][u] && !visited[u]) {
98                 visited[u] = 1;
99                 visited_count++;
100                 q[qe++] = u;
101             }
102         }
103     }
104
105     // Проверяем, все ли вершины из ребер были посещены
106     for (int i = 0; i < m; i++) {
107         if (!visited[edges[i].u] || !visited[edges[i].v]) {

```

```

108     return 0; // Нашли вершину, которая не была посещена
109 }
110 }
111
112 return visited_count == vertex_count; // Все вершины должны быть посещены
113 }
114 // Доставка UDP с 3 попытками отправки
115 void send_udp_reliable(int sock, struct sockaddr_in *addr, char *data,
116     int size) {
117     socklen_t len = sizeof(*addr);
118     char ack[8];
119
120     for (int attempt = 1; attempt <= 3; attempt++) {
121         printf("[CLIENT][UDP] Отправка пакета попытка( %d)...\n", attempt);
122         sendto(sock, data, size, 0, (struct sockaddr*)addr, len);
123         // ждём подтверждения
124         struct timeval tv = {3, 0};
125         fd_set fds;
126         FD_ZERO(&fds);
127         FD_SET(sock, &fds);
128         int r = select(sock + 1, &fds, NULL, NULL, &tv);
129         if (r > 0) {
130             recvfrom(sock, ack, sizeof(ack), 0, NULL, NULL);
131             printf("[CLIENT][UDP] Получен ACK от сервера\n");
132             return;
133         }
134         printf("[CLIENT][UDP] ACK не получен\n");
135     }
136     printf("Потеряна связь с сервером\n");
137     exit(0);
138 }
139
140 int main(int argc, char *argv[]) {
141     if (argc != 4) {
142         printf("Использование: %s <IP> <tcp|udp> <PORT>\n", argv[0]);
143         exit(0);
144     }
145
146     char *ip = argv[1];
147     int is_tcp = (strcmp(argv[2], "tcp") == 0);
148     int port = atoi(argv[3]);
149
150     int sock = socket(AF_INET, is_tcp ? SOCK_STREAM : SOCK_DGRAM, 0);
151     if (sock < 0) die("socket");
152
153     struct sockaddr_in serv;
154     serv.sin_family = AF_INET;
155     serv.sin_port = htons(port);
156     inet_pton(AF_INET, ip, &serv.sin_addr);
157
158     if (is_tcp) {
159         printf("[CLIENT][TCP] Подключение к серверу...\n");
160         if (connect(sock, (struct sockaddr*)&serv, sizeof(serv)) < 0)
161             die("connect");
162         printf("[CLIENT][TCP] Соединение установлено\n");
163     }

```

```

163
164 while (1) {
165     printf("\Выберите источник данных:\n");
166     printf("1 - ввод с клавиатуры\n");
167     printf("2 - ввод из файла\n");
168     printf("Введите 'exit' для выхода\n> ");
169
170     char mode_input[32];
171     fgets(mode_input, sizeof(mode_input), stdin);
172
173     if (strncmp(mode_input, "exit", 4) == 0)
174         break;
175
176     int mode = atoi(mode_input);
177     if (mode != 1 && mode != 2) {
178         printf("Некорректный выбор\n");
179         continue;
180     }
181
182     int m, s, t;
183     Edge edges[MAX_EDGES];
184
185     if (mode == 1) {
186         // ===== ВВОД С КЛАВИАТУРЫ =====
187         printf("Введите количество рёбер: ");
188         char bufm[32];
189         fgets(bufm, sizeof(bufm), stdin);
190         m = atoi(bufm);
191
192         if (m <= 0) {
193             printf("Некорректное значение\n");
194             continue;
195         }
196
197         printf("Введите %d рёбер (u v w):\n", m);
198         for (int i = 0; i < m; i++)
199             scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
200
201         printf("Начальная и конечная вершины: ");
202         scanf("%d %d", &s, &t);
203         getchar(); // очистка stdin
204
205     } else {
206         // ===== ВВОД ИЗ ФАЙЛА =====
207         char filename[256];
208         printf("Введите имя файла: ");
209         fgets(filename, sizeof(filename), stdin);
210         filename[strlen(filename)] = '\0';
211
212         FILE *f = fopen(filename, "r");
213         if (!f) {
214             printf("Ошибка: не удалось открыть файл\n");
215             continue;
216         }
217
218         if (fscanf(f, "%d", &m) != 1 || m <= 0) {

```

```

219     printf("Ошибка: неверный формат файла\n");
220     fclose(f);
221     continue;
222 }
223
224 for (int i = 0; i < m; i++) {
225     if (fscanf(f, "%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w)
226 != 3) {
227         printf("Ошибка: неверный формат рёбер\n");
228         fclose(f);
229         continue;
230     }
231
232     if (fscanf(f, "%d %d", &s, &t) != 2) {
233         printf("Ошибка: неверный формат стартфиниш/\n");
234         fclose(f);
235         continue;
236     }
237
238     fclose(f);
239     printf("[CLIENT] Данные успешно считаны из файла '%s'\n", filename);
240 }
241 int vertex_count = count_vertices(edges, m);
242
243 // Проверка количества вершин
244 if (vertex_count < 6 || vertex_count > 100) {
245     printf("[CLIENT] Ошибка: граф должен содержать от 6 до 100 вершин!\n");
246     printf("Текущее количество вершин: %d\n", vertex_count);
247     printf("Пожалуйста, введите данные снова.\n");
248     continue;
249 }
250
251 // Проверка количества рёбер
252 if (!check_edge_count(m, vertex_count)) {
253     if (m < 6) {
254         printf("[CLIENT] Ошибка: граф должен содержать не менее 6 рёбер!\n");
255     } else if (m > 100) {
256         printf("[CLIENT] Ошибка: граф должен содержать не более 100 рёбер!\n");
257     }
258     printf("Текущее количество рёбер: %d\n", m);
259     printf("Пожалуйста, введите данные снова.\n");
260     continue;
261 }
262
263 // Проверка связности графа
264 if (!check_connected(edges, m, vertex_count)) {
265     printf("[CLIENT] Ошибка: граф несвязный!\n");
266     printf("Введите другой граф.\n");
267     continue;
268 }
269 // Формирование пакета
270 int packet[3 + MAX_EDGES * 3];
271 packet[0] = m;
272 packet[1] = s;

```

```

273     packet[2] = t;
274
275     for (int i = 0; i < m; i++) {
276         packet[3 + i*3] = edges[i].u;
277         packet[4 + i*3] = edges[i].v;
278         packet[5 + i*3] = edges[i].w;
279     }
280
281     int packet_size = sizeof(int) * (3 + m * 3);
282     // Отправка
283     if (is_tcp) {
284         printf("[CLIENT][TCP] Отправка данных серверу...\n");
285         send(sock, packet, packet_size, 0);
286     } else {
287         printf("[CLIENT][UDP] Надёжная отправка данных...\n");
288         send_udp_reliable(sock, &serv, (char*)packet, packet_size);
289     }
290     // Получение ответа
291     char response[BUF_SIZE];
292     int r = is_tcp ?
293     recv(sock, response, sizeof(response), 0) :
294     recvfrom(sock, response, sizeof(response), 0, NULL, NULL);
295
296     if (r <= 0) {
297         printf("[CLIENT] Ошибка получения ответа\n");
298         continue;
299     }
300
301     printf("[CLIENT] Ответ сервера: %s\n", response);
302 }
303
304 close(sock);
305 return 0;
306 }

```


Приложение 2. Исходный код сервера

Листинг 7. Исходный код сервера

```
1 // server.c - сервер: TCP/UDP, многопоточность, Дейкстра, служебные сообщения,  
   надёжная доставка UDP  
2 #include <stdio.h>  
3 #include <stdlib.h>  
4 #include <string.h>  
5 #include <unistd.h>  
6 #include <pthread.h>  
7 #include <arpa/inet.h>  
8 #include <sys/socket.h>  
9  
10 #define MAX_EDGES 128  
11 #define MAXV 128  
12 #define INF 1000000000  
13 #define BUF_SIZE 2048  
14  
15 typedef struct {  
16     int u, v, w;  
17 } Edge;  
18  
19 typedef struct {  
20     int sock;  
21     struct sockaddr_in addr;  
22     int is_tcp;  
23 } ClientInfo;  
24 // Аварийное завершение  
25 void die(const char *msg) {  
26     perror(msg);  
27     exit(EXIT_FAILURE);  
28 }  
29 // Алгоритм Дейкстры  
30 void dijkstra(int n, int s, int t, Edge *edges, int m, char *out) {  
31     int g[MAXV][MAXV];  
32     for (int i = 0; i < n; i++)  
33         for (int j = 0; j < n; j++)  
34             g[i][j] = INF;  
35  
36     for (int i = 0; i < m; i++) {  
37         int u = edges[i].u, v = edges[i].v, w = edges[i].w;  
38         g[u][v] = g[v][u] = w;  
39     }  
40  
41     int dist[MAXV], used[MAXV], parent[MAXV];  
42     for (int i = 0; i < n; i++) {  
43         dist[i] = INF;  
44         used[i] = 0;  
45         parent[i] = -1;  
46     }  
47  
48     dist[s] = 0;  
49  
50     for (int it = 0; it < n; it++) {  
51         int v = -1;
```

```

52     for (int i = 0; i < n; i++)
53         if (!used[i] && (v == -1 || dist[i] < dist[v]))
54             v = i;
55
56     if (dist[v] == INF) break;
57     used[v] = 1;
58
59     for (int u = 0; u < n; u++) {
60         if (g[v][u] < INF && dist[u] > dist[v] + g[v][u]) {
61             dist[u] = dist[v] + g[v][u];
62             parent[u] = v;
63         }
64     }
65 }
66
67 // восстановление пути
68 int path[MAXV], len = 0;
69 for (int v = t; v != -1; v = parent[v])
70     path[len++] = v;
71
72 sprintf(out, "Длина: %d; Путь: ", dist[t]);
73 for (int i = len - 1; i >= 0; i--) {
74     char tmp[16];
75     sprintf(tmp, "%d ", path[i]);
76     strcat(out, tmp);
77 }
78 }
79 // обработчик клиентских запросов
80 void *client_thread(void *arg) {
81     ClientInfo *ci = (ClientInfo*)arg;
82     while (1) {
83         int buf[3 + MAX_EDGES * 3];
84         struct sockaddr_in from;
85         socklen_t fl = sizeof(from);
86
87         int r = ci->is_tcp ?
88             recv(ci->sock, buf, sizeof(buf), 0) :
89             recvfrom(ci->sock, buf, sizeof(buf), 0, (struct sockaddr*)&from, &
90 fl);
91
92         if (r <= 0) {
93             if (ci->is_tcp) printf("[SERVER][TCP] Клиент отключился\n");
94             return NULL;
95         }
96
97         if (!ci->is_tcp) {
98             // отправляем ACK
99             printf("[SERVER][UDP] Пакет получен, отправляю ACK\n");
100             sendto(ci->sock, "ACK", 4, 0, (struct sockaddr*)&from, fl);
101         }
102
103         int m = buf[0];
104         int s = buf[1];
105         int t = buf[2];
106
107         Edge edges[MAX_EDGES];

```

```

107     int maxv = 0;
108
109     for (int i = 0; i < m; i++) {
110         edges[i].u = buf[3 + i*3];
111         edges[i].v = buf[4 + i*3];
112         edges[i].w = buf[5 + i*3];
113
114         if (edges[i].u > maxv) maxv = edges[i].u;
115         if (edges[i].v > maxv) maxv = edges[i].v;
116     }
117     if (s < 0 || t < 0 || s > maxv || t > maxv) {
118         fprintf(stderr, "[SERVER] Неверная начальная (%d) или конечная
вершины (%d) графа\n",
119             s, t, maxv);
120         continue;
121     }
122
123     if (maxv + 1 > 100) {
124         fprintf(stderr, "[SERVER] Слишком много вершин: %d\n", maxv);
125         continue;
126     }
127     char response[BUF_SIZE] = {0};
128     dijkstra(maxv + 1, s, t, edges, m, response);
129
130     if (ci->is_tcp) {
131         printf("[SERVER][TCP] Отправка ответа клиенту\n");
132         send(ci->sock, response, strlen(response)+1, 0);
133     } else {
134         printf("[SERVER][UDP] Отправка ответа клиенту\n");
135         sendto(ci->sock, response, strlen(response)+1, 0,
136             (struct sockaddr*)&from, fl);
137     }
138 }
139
140 return NULL;
141 }
142
143 int main(int argc, char *argv[]) {
144     if (argc != 3) {
145         printf("Использование: %s <tcp|udp> <PORT>\n", argv[0]);
146         exit(0);
147     }
148
149     int is_tcp = strcmp(argv[1], "tcp") == 0;
150     int port = atoi(argv[2]);
151
152     int sock = socket(AF_INET, is_tcp ? SOCK_STREAM : SOCK_DGRAM, 0);
153     if (sock < 0) die("socket");
154
155     struct sockaddr_in serv;
156     serv.sin_family = AF_INET;
157     serv.sin_port = htons(port);
158     serv.sin_addr.s_addr = INADDR_ANY;
159
160     if (bind(sock, (struct sockaddr*)&serv, sizeof(serv)) < 0)
161         die("bind");

```

```

162
163     printf("[SERVER] Запущен. Протокол: %s, порт %d\n",
164     is_tcp ? "TCP" : "UDP", port);
165
166     if (is_tcp) {
167         listen(sock, 10);
168         while (1) {
169             struct sockaddr_in cli;
170             socklen_t cl = sizeof(cli);
171             int csock = accept(sock, (struct sockaddr*)&cli, &cl);
172
173             printf("[SERVER][TCP] Новое подключение!\n");
174
175             ClientInfo *ci = malloc(sizeof(ClientInfo));
176             ci->sock = csock;
177             ci->is_tcp = 1;
178
179             pthread_t th;
180             pthread_create(&th, NULL, client_thread, ci);
181             pthread_detach(th);
182         }
183     } else {
184         ClientInfo ci;
185         ci.sock = sock;
186         ci.is_tcp = 0;
187
188         client_thread(&ci);
189     }
190
191     return 0;
192 }
193

```

Приложение 3. Исходный код тестирующего приложения

Листинг 8. Исходный код тестирующего приложения

```
1 #!/usr/bin/expect -f
2 set timeout 10
3 set client [lindex $argv 0]
4 set server [lindex $argv 1]
5
6 # Глобальные счетчики для статистики
7 set total_tests 0
8 set passed_tests 0
9
10 # Создаем тестовые файлы данных
11 proc create_test_files {} {
12     # Файл для нормального тестирования середина( ОДЗ)
13     set fp [open "test_mid.txt" w]
14     puts $fp "8"
15     puts $fp "0 1 5"
16     puts $fp "0 2 3"
17     puts $fp "1 2 2"
18     puts $fp "1 3 6"
19     puts $fp "2 3 7"
20     puts $fp "3 4 4"
21     puts $fp "4 5 3"
22     puts $fp "5 6 2"
23     puts $fp "0 6"
24     close $fp
25
26     # Файл с нижней границей ОДЗ (6 ребер)
27     set fp [open "test_lower.txt" w]
28     puts $fp "6"
29     puts $fp "0 1 1"
30     puts $fp "1 2 2"
31     puts $fp "2 3 3"
32     puts $fp "3 4 4"
33     puts $fp "4 5 5"
34     puts $fp "5 0 6"
35     puts $fp "0 3"
36     close $fp
37     set fp [open "test_upper.txt" w]
38     puts $fp "100"
39     for {set i 0} {$i < 99} {incr i} {
40         puts $fp "$i [expr {$i+1}] 1"
41     }
42     puts $fp "9 0 1"
43     puts $fp "0 99"
44     close $fp
45     set fp [open "test_upper_UP.txt" w]
46     puts $fp "101"
47     for {set i 0} {$i < 100} {incr i} {
48         puts $fp "$i [expr {$i+1}] 1"
49     }
50     puts $fp "100 0 1"
```

```

51 puts $fp "0 100"
52 close $fp
53 puts "Созданы тестовые файлы"
54 }
55
56 # Освобождение порта
57 proc free_port {port} {
58     catch {exec fuser -k $port/tcp >/dev/null 2>&1}
59     catch {exec fuser -k $port/udp >/dev/null 2>&1}
60     after 100
61 }
62
63 # Простая функция запуска сервера
64 proc start_simple_server {protocol port} {
65     global server
66
67     # Освобождаем порт
68     free_port $port
69
70     # Запускаем сервер в фоне
71     exec $server $protocol $port > /tmp/server_${protocol}_${port}.log
72     2>&1 &
73
74     # Даем время на запуск
75     after 1000
76
77     # Проверяем, что процесс запущен
78     set pid [exec ps aux | grep "$server $protocol $port" | grep -v grep |
79         awk "{print \$2}"]
80
81     if {$pid != ""} {
82         puts "Сервер $protocol запущен на порту $port (PID: $pid)"
83         return $pid
84     } else {
85         puts "Не удалось запустить сервер $protocol на порту $port"
86         return ""
87     }
88 }
89
90 # Остановка сервера
91 proc stop_server {pid} {
92     if {$pid != ""} {
93         catch {exec kill $pid 2>/dev/null}
94         after 500
95         catch {exec kill -9 $pid 2>/dev/null}
96     }
97 }
98
99 # Функция для вывода результата теста и обновления статистики
100 proc report_test {result message} {
101     global total_tests passed_tests
102
103     incr total_tests
104     if {[string equal -nocase $result "true"]} {
105         incr passed_tests
106         puts "True: $message"
107     }
108 }

```

```

105 } else {
106     puts "False: $message"
107 }
108 }
109
110 # Тест TCP
111 proc test_tcp_simple {} {
112     global client
113
114     puts "\n=== ТЕСТ 1: Работа по протоколу TCP ==="
115
116     # Запускаем сервер TCP
117     set pid [start_simple_server tcp 8888]
118
119     if {$pid == ""} {
120         puts "Пропускаем TCP тесты"
121         return
122     }
123
124     # Тест 1.1: Ввод с клавиатуры (TCP)
125     puts "\n--- 1.1: Ввод с клавиатуры (TCP) ---"
126
127     set test_passed 0
128     spawn $client 127.0.0.1 tcp 8888
129
130     expect {
131         "> " {
132             send -- "1\r"
133             set test_passed 1
134         }
135         timeout {
136             report_test "false" "TCP с вводом с клавиатуры"
137             catch {close}
138             catch {wait}
139             set test_passed 0
140         }
141     }
142
143     if {$test_passed} {
144         expect "количество рёбер: "
145         send -- "8\r"
146
147         expect "рёбер (u v w):"
148         send -- "0 1 5\r"
149         send -- "0 2 3\r"
150         send -- "1 2 2\r"
151         send -- "1 3 6\r"
152         send -- "2 3 7\r"
153         send -- "3 4 4\r"
154         send -- "4 5 3\r"
155         send -- "5 6 2\r"
156
157         expect "вершины: "
158         send -- "0 6\r"
159
160         expect {

```

```

161     "Ответ сервера: Длина: 19; Путь: 0 2 3 4 5 6" {
162         report_test "true" "TCP с вводом с клавиатуры"
163     }
164     timeout {
165         report_test "false" "TCP с вводом с клавиатуры"
166     }
167 }
168
169 send -- "exit\r"
170 expect eof
171 }
172
173 # Перезапускаем сервер для следующего теста
174 stop_server $pid
175 set pid [start_simple_server tcp 8888]
176
177 if {$pid == ""} {
178     puts "Не удалось перезапустить сервер"
179     set test_passed 0
180 } else {
181     set test_passed 1
182 }
183
184 # Тест 1.2: Ввод из файла (TCP)
185 if {$test_passed} {
186     puts "\n--- 1.2: Ввод из файла (TCP) ---"
187     spawn $client 127.0.0.1 tcp 8888
188
189     expect {
190         "> " {
191             send -- "2\r"
192             set test_passed 1
193         }
194         timeout {
195             report_test "false" "TCP с файловым вводом"
196             set test_passed 0
197             catch {close}
198             catch {wait}
199         }
200     }
201
202     if {$test_passed} {
203         expect "имя файла: "
204         send -- "test_mid.txt\r"
205
206         expect {
207             "Ответ сервера: Длина: 19; Путь: 0 2 3 4 5 6" {
208                 report_test "true" "TCP с файловым вводом"
209             }
210             timeout {
211                 report_test "false" "TCP с файловым вводом"
212             }
213         }
214
215         send -- "exit\r"
216         expect eof

```



```

217     }
218 }
219
220 # Тест 1.3: Несвязный граф
221 stop_server $pid
222 set pid [start_simple_server tcp 8894]
223
224 if {$pid == ""} {
225     puts "Не удалось запустить сервер для теста несвязного графа"
226     set test_passed 0
227 } else {
228     set test_passed 1
229 }
230
231 if {$test_passed} {
232     puts "\n--- 1.3: Несвязный граф (TCP) ---"
233     spawn $client 127.0.0.1 tcp 8894
234
235     expect {
236         "> " {
237             send -- "1\r"
238             set test_passed 1
239         }
240         timeout {
241             report_test "false" "Несвязный граф"
242             set test_passed 0
243             catch {close}
244             catch {wait}
245         }
246     }
247
248     if {$test_passed} {
249         expect "количество рёбер: "
250         send -- "6\r"
251
252         expect "рёбер (u v w):"
253         send -- "0 1 1\r"
254         send -- "1 2 2\r"
255         send -- "2 0 3\r"
256         send -- "3 4 4\r"
257         send -- "4 5 5\r"
258         send -- "5 3 6\r"
259
260         expect "вершины: "
261         send -- "0 4\r"
262
263         expect {
264             "Ошибка: граф несвязный" {
265                 report_test "true" "Несвязный граф"
266             }
267             timeout {
268                 report_test "false" "Несвязный граф"
269             }
270         }
271
272         send -- "exit\r"

```

```

273     expect eof
274 }
275 }
276
277 if {$pid != ""} {
278     stop_server $pid
279 }
280
281 puts "\nTCP тесты завершены"
282 }
283
284 # Тест UDP
285 proc test_udp_simple {} {
286     global client
287
288     puts "\n=== ТЕСТ 2: Работа по протоколу UDP ==="
289
290     # Запускаем сервер UDP
291     set pid [start_simple_server udp 8889]
292
293     if {$pid == ""} {
294         puts "Пропускаем UDP тесты"
295         return
296     }
297
298     # Тест 2.1: Нормальная работа UDP
299     puts "\n--- 2.1: Нормальная работа UDP ---"
300
301     set test_passed 0
302     spawn $client 127.0.0.1 udp 8889
303
304     expect {
305         "> " {
306             send -- "1\r"
307             set test_passed 1
308         }
309         timeout {
310             report_test "false" "UDP с клавиатурным вводом"
311             catch {close}
312             catch {wait}
313         }
314     }
315
316     if {$test_passed} {
317         expect "количество рёбер: "
318         send -- "7\r"
319
320         expect "рёбер (u v w):"
321         send -- "0 1 1\r"
322         send -- "1 2 2\r"
323         send -- "2 3 3\r"
324         send -- "3 4 4\r"
325         send -- "4 5 5\r"
326         send -- "5 6 6\r"
327         send -- "6 0 7\r"
328

```

```

329 expect "вершины: "
330 send -- "0 4\r"
331
332 expect {
333     "Ответ сервера: Длина: 10; Путь: 0 1 2 3 4" {
334         report_test "true" "UDP с клавиатурным вводом"
335         send -- "exit\r"
336         expect eof
337     }
338     "Потеряна связь с сервером" {
339         report_test "false" "UDP с клавиатурным вводом"
340         expect eof
341     }
342     timeout {
343         report_test "false" "UDP с клавиатурным вводом"
344         catch {send -- "exit\r"}
345         catch {expect eof}
346     }
347 }
348 }
349
350 # Останавливаем сервер для теста недоступности
351 stop_server $pid
352
353 # Тест 2.2: Проверка недоступности сервера
354 puts "\n--- 2.2: Проверка недоступности сервера ---"
355 after 2000
356
357 # Пытаемся подключиться к несуществующему серверу
358 set test_passed 0
359 spawn $client 127.0.0.1 udp 8889
360
361 expect {
362     "> " {
363         send -- "1\r"
364         set test_passed 1
365     }
366     timeout {
367         report_test "false" "Недоступный сервер UDP"
368         catch {close}
369         catch {wait}
370     }
371 }
372
373 if {$test_passed} {
374     expect "количество рёбер: "
375     send -- "7\r"
376
377     expect "рёбер (u v w):"
378     send -- "0 1 1\r"
379     send -- "1 2 2\r"
380     send -- "2 3 3\r"
381     send -- "3 4 4\r"
382     send -- "4 5 5\r"
383     send -- "5 6 6\r"
384     send -- "6 0 7\r"

```

```

385
386 expect "вершины: "
387 send -- "0 4\r"
388
389 expect {
390     "Потеряна связь с сервером" {
391         report_test "true" "Недоступный сервер UDP"
392         expect eof
393     }
394     timeout {
395         report_test "false" "Недоступный сервер UDP"
396         catch {send -- "exit\r"}
397         catch {expect eof}
398     }
399 }
400 }
401
402 puts "UDP тесты завершены"
403 }
404
405 # Тест граничных значений
406 proc test_boundaries_simple {} {
407     global client
408
409     puts "\n=== ТЕСТ 3: Граничные значения ОДЗ ==="
410
411     # Используем разные порты для каждого подтеста
412     set ports {8890 8891 8892 8893 8895}
413     set port_index 0
414
415     # Тест 3.1: Нижняя граница - 1 (5 вершин)
416     puts "\n--- 3.1: Нижняя граница - 1 (5 вершин) ---"
417
418     set pid [start_simple_server tcp [lindex $ports $port_index]]
419     incr port_index
420
421     if {$pid != ""} {
422         spawn $client 127.0.0.1 tcp [lindex $ports [expr {$port_index - 1}]]
423
424         expect {
425             "> " {
426                 send -- "1\r"
427                 set test_passed 1
428             }
429             timeout {
430                 report_test "false" "Нижняя граница -1"
431                 set test_passed 0
432                 catch {close}
433                 catch {wait}
434             }
435         }
436
437         if {$test_passed} {
438             expect "количество рёбер: "
439             send -- "5\r"
440

```

```

441     expect "рёбер (u v w):"
442     send -- "0 1 1\r"
443     send -- "1 2 2\r"
444     send -- "2 3 3\r"
445     send -- "3 4 4\r"
446     send -- "4 0 5\r"
447
448     expect "вершины: "
449     send -- "0 3\r"
450
451     expect {
452         "Ошибка: граф должен содержать от 6 до 100 вершин!" {
453             report_test "true" "Нижняя граница -1"
454         }
455         timeout {
456             report_test "false" "Нижняя граница -1"
457         }
458     }
459
460     send -- "exit\r"
461     expect eof
462 }
463
464 stop_server $pid
465 } else {
466     report_test "false" "Граничные-3.1: Нижняя граница -1"
467 }
468
469 # Тест 3.2: Нижняя граница (6 вершин)
470 puts "\n--- 3.2: Нижняя граница (6 вершин) ---"
471
472 set pid [start_simple_server tcp [lindex $ports $port_index]]
473 incr port_index
474
475 if {$pid != ""} {
476     spawn $client 127.0.0.1 tcp [lindex $ports [expr {$port_index - 1}]]
477
478     expect {
479         "> " {
480             send -- "2\r"
481             set test_passed 1
482         }
483         timeout {
484             report_test "false" "Нижняя граница"
485             set test_passed 0
486             catch {close}
487             catch {wait}
488         }
489     }
490
491     if {$test_passed} {
492         expect "имя файла: "
493         send -- "test_lower.txt\r"
494
495         expect {
496             "Ответ сервера: Длина: 6; Путь: 0 1 2 3" {

```

```

497         report_test "true" "Нижняя граница"
498     }
499     timeout {
500         report_test "false" "Нижняя граница"
501     }
502 }
503
504     send -- "exit\r"
505     expect eof
506 }
507
508     stop_server $pid
509 } else {
510     report_test "false" "Нижняя граница"
511 }
512
513 # Тест 3.3: Середина ОДЗ
514 puts "\n--- 3.3: Середина ОДЗ ---"
515
516 set pid [start_simple_server tcp [lindex $ports $port_index]]
517 incr port_index
518
519 if {$pid != ""} {
520     spawn $client 127.0.0.1 tcp [lindex $ports [expr {$port_index - 1}]]
521
522     expect {
523         "> " {
524             send -- "2\r"
525             set test_passed 1
526         }
527         timeout {
528             report_test "false" "Середина ОДЗ"
529             set test_passed 0
530             catch {close}
531             catch {wait}
532         }
533     }
534
535     if {$test_passed} {
536         expect "имя файла: "
537         send -- "test_mid.txt\r"
538
539         expect {
540             "Ответ сервера: Длина: 19; Путь: 0 2 3 4 5 6" {
541                 report_test "true" "Середина ОДЗ"
542             }
543             timeout {
544                 report_test "false" "Середина ОДЗ"
545             }
546         }
547
548         send -- "exit\r"
549         expect eof
550     }
551
552     stop_server $pid

```

```

553 } else {
554     report_test "false" "Середина 0ДЗ"
555 }
556
557 # Тест 3.4: Верхняя граница (100 вершин)
558 puts "\n--- 3.4: Верхняя граница (100 вершин) ---"
559
560 set pid [start_simple_server tcp [lindex $ports $port_index]]
561 incr port_index
562
563 if {$pid != ""} {
564     spawn $client 127.0.0.1 tcp [lindex $ports [expr {$port_index - 1}]]
565
566     expect {
567         "> " {
568             send -- "2\r"
569             set test_passed 1
570         }
571         timeout {
572             report_test "false" "Верхняя граница"
573             set test_passed 0
574             catch {close}
575             catch {wait}
576         }
577     }
578
579     if {$test_passed} {
580         expect "имя файла: "
581         send -- "test_upper.txt\r"
582
583         expect {
584             "Ответ сервера: Длина: 91; Путь: 0 9 10 11 12 13 14 15 16 17 18 19
585             20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42
586             43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65
587             66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88
588             89 90 91 92 93 94 95 96 97 98 99 " {
589                 report_test "true" "Верхняя граница"
590             }
591             timeout {
592                 report_test "false" "Верхняя граница"
593             }
594         }
595
596         send -- "exit\r"
597         expect eof
598     }
599
600     stop_server $pid
601 } else {
602     report_test "false" "Верхняя граница"
603 }
604
605 # Тест 3.5: Верхняя граница +1 (101 вершина)
606 puts "\n--- 3.5: Верхняя граница + 1 ---"
607
608 set pid [start_simple_server tcp [lindex $ports $port_index]]

```

```

605
606 if {$pid != ""} {
607     spawn $client 127.0.0.1 tcp [lindex $ports $port_index]
608
609     expect {
610         "> " {
611             send -- "2\r"
612             set test_passed 1
613         }
614         timeout {
615             report_test "false" "Верхняя граница +1"
616             set test_passed 0
617             catch {close}
618             catch {wait}
619         }
620     }
621
622     if {$test_passed} {
623         expect "имя файла: "
624         send -- "test_upper_UP.txt\r"
625
626         expect {
627             "Ошибка: граф должен содержать от 6 до 100 вершин!" {
628                 report_test "true" "Верхняя граница +1"
629             }
630             timeout {
631                 report_test "false" "Верхняя граница +1"
632             }
633         }
634
635         send -- "exit\r"
636         expect eof
637     }
638
639     stop_server $pid
640 } else {
641     report_test "false" "Верхняя граница +1"
642 }
643
644 puts "Тесты граничных значений завершены"
645 }
646
647 # Функция для вывода статистики
648 proc print_statistics {} {
649     global total_tests passed_tests
650
651     puts ""
652     puts [string repeat "=" 50]
653     puts "Статистика"
654     set expected_total 10
655     puts "Ожидалось тестов: $expected_total"
656     puts "Выполнено тестов: $total_tests из 10"
657     puts "Пройдено тестов: $passed_tests из 10"
658     puts "Провалено тестов: [expr {$total_tests - $passed_tests}] из 10"
659     puts [string repeat "=" 50]
660 }

```



```

661
662 # Основная процедура
663 proc main {} {
664     global client server argv total_tests passed_tests
665
666     # Инициализируем счетчики
667     set total_tests 0
668     set passed_tests 0
669
670     if {[llength $argv] != 2} {
671         puts "Использование: $argv0 <client_executable> <server_executable>"
672         puts "Пример: ./test_app.sh ./client ./server"
673         exit 1
674     }
675
676     set client [lindex $argv 0]
677     set server [lindex $argv 1]
678
679     # Проверяем существование исполняемых файлов
680     if {[file executable $client]} {
681         puts "Ошибка: $client не существует или не исполняемый"
682         exit 1
683     }
684
685     if {[file executable $server]} {
686         puts "Ошибка: $server не существует или не исполняемый"
687         exit 1
688     }
689
690     puts "Начинаем тестирование клиентсерверного- приложения"
691     puts "Клиент: $client"
692     puts "Сервер: $server"
693
694     # Освобождаем порты
695     for {set port 8888} {$port <= 8900} {incr port} {
696         free_port $port
697     }
698
699     # Создаем тестовые файлы
700     create_test_files
701
702     puts "\n"
703
704     # Запускаем тесты
705     test_tcp_simple
706     test_udp_simple
707     test_boundaries_simple
708
709     # Выводим статистику
710     print_statistics
711
712     # Удаляем временные файлы
713     file delete test_mid.txt
714     file delete test_lower.txt
715     file delete test_upper.txt
716     file delete test_upper_UP.txt

```

```
717  
718     puts "Временные файлы удалены"  
719 }  
720  
721 # Запуск главной процедуры  
722 main
```

Приложение 4. Протоколы тестирования

На рисунках 5- 7 представлены тесты по протоколу TCP: ввод с клавиатуры, ввод из файла и обработка несвязного графа.

```
user@debian13:~/socket_example$ ./tests.sh ./client ./server
Начинаем тестирование клиент-серверного приложения
Клиент: ./client
Сервер: ./server
Созданы тестовые файлы

=== ТЕСТ 1: Работа по протоколу TCP ===
True: Сервер tcp запущен на порту 8888 (PID: 29331)

--- 1.1: Ввод с клавиатуры (TCP) ---
spawn ./client 127.0.0.1 tcp 8888
[CLIENT][TCP] Подключение к серверу...
[CLIENT][TCP] Соединение установлено

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 1
Введите количество рёбер: 8
Введите 8 рёбер (u v w):
0 1 5
0 2 3
1 2 2
1 3 6
2 3 7
3 4 4
4 5 3
5 6 2
Начальная и конечная вершины: 0 6
[CLIENT][TCP] Отправка данных серверу...
[CLIENT] Ответ сервера: Длина: 19; Путь: 0 2 3 4 5 6
True: TCP тест с клавиатурным вводом пройден

exit
```

Рис. 5. Ввод с клавиатуры

```
Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> Сервер остановлен (PID: 29331)
True: Сервер tcp запущен на порту 8888 (PID: 29366)

--- 1.2: Ввод из файла (TCP) ---
spawn ./client 127.0.0.1 tcp 8888
[CLIENT][TCP] Подключение к серверу...
[CLIENT][TCP] Соединение установлено

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 2
Введите имя файла: test_mid.txt
[CLIENT] Данные успешно считаны из файла 'test_mid.txt'
[CLIENT][TCP] Отправка данных серверу...
[CLIENT] Ответ сервера: Длина: 19; Путь: 0 2 3 4 5 6

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
>
True: TCP тест с файловым вводом пройден
exit
Сервер остановлен (PID: 29366)
True: Сервер tcp запущен на порту 8894 (PID: 29379)
```

Рис. 6. Ввод из файла

```
=== 1.3: Несвязный граф (TCP) ===
spawn ./client 127.0.0.1 tcp 8894
[CLIENT][TCP] Подключение к серверу...
[CLIENT][TCP] Соединение установлено

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 1
Введите количество рёбер: 6
Введите 6 рёбер (u v w):
0 1 1
1 2 2
2 0 3
3 4 4
4 5 5
5 3 6
Начальная и конечная вершины: 0 4
[CLIENT] Ошибка: граф несвязный!
Введите другой граф.

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> True: Тест несвязного графа пройден
exit
Сервер остановлен (PID: 29379)

TCP тесты завершены
```

Рис. 7. Несвязный граф

На рисунках 8- 9 представлены тесты по протоколу UDP: нормальная работа UDP и проверка недоступности сервера.

```
=== ТЕСТ 2: Работа по протоколу UDP ===
True: Сервер udp запущен на порту 8889 (PID: 29392)

--- Подтест 2.1: Нормальная работа UDP ---
spawn ./client 127.0.0.1 udp 8889

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 1
Введите количество рёбер: 7
Введите 7 рёбер (u v w): 0 1 1
1 2 2
2 3 3
3 4 4
4 5 5
5 6 6
6 0 7

Начальная и конечная вершины: 0 4
[CLIENT][UDP] Надёжная отправка данных...
[CLIENT][UDP] Отправка пакета (попытка 1)...
[CLIENT][UDP] Получен ACK от сервера
[CLIENT] Ответ сервера: Длина: 10; Путь: 0 1 2 3 4 True: UDP тест с клавиатурным вводом пройден
exit

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> Сервер остановлен (PID: 29392)
```

Рис. 8. Нормальная работа UDP

```
--- Подтест 2.2: Проверка недоступности сервера ---
spawn ./client 127.0.0.1 udp 8889

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 1
Введите количество рёбер: 7
Введите 7 рёбер (u v w):
0 1 1
1 2 2
2 3 3
3 4 4
4 5 5
5 6 6
6 0 7

Начальная и конечная вершины: 0 4
[CLIENT][UDP] Надёжная отправка данных...
[CLIENT][UDP] Отправка пакета (попытка 1)...
[CLIENT][UDP] ACK не получен
[CLIENT][UDP] Отправка пакета (попытка 2)...
[CLIENT][UDP] ACK не получен
[CLIENT][UDP] Отправка пакета (попытка 3)...
[CLIENT][UDP] ACK не получен
Потеряна связь с сервером
True: UDP тест с недоступным сервером пройден
UDP тесты завершены
```

Рис. 9. Проверка недоступности сервера

На рисунках 10- 14 представлены тесты по проверке работоспособности программы при задании разного количества вершин.

```
=== ТЕСТ 3: Граничные значения ОДЗ ===

--- Подтест 3.1: Нижняя граница - 1 (5 вершин) ---
True: Сервер tcp запущен на порту 8890 (PID: 29494)
spawn ./client 127.0.0.1 tcp 8890
[CLIENT][TCP] Подключение к серверу...
[CLIENT][TCP] Соединение установлено

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 1
Введите количество рёбер: 5
Введите 5 рёбер (u v w):
0 1 1
1 2 2
2 3 3
3 4 4
4 0 5
Начальная и конечная вершины: 0 3
[CLIENT] Ошибка: граф должен содержать от 6 до 100 вершин!
Текущее количество вершин: 5
Пожалуйста, введите данные снова.

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> True: Тест нижней границы -1 пройден
exit
Сервер остановлен (PID: 29494)
```

Рис. 10. Граф с количеством вершин, на единицу меньше нижней границы ОДЗ

```

--- Подтест 3.2: Нижняя граница (6 вершин) ---
True: Сервер tcp запущен на порту 8891 (PID: 29543)
spawn ./client 127.0.0.1 tcp 8891
[CLIENT][TCP] Подключение к серверу...
[CLIENT][TCP] Соединение установлено

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 2
Введите имя файла: test_lower.txt
[CLIENT] Данные успешно считаны из файла 'test_lower.txt'
[CLIENT][TCP] Отправка данных серверу...
[CLIENT] Ответ сервера: Длина: 6; Путь: 0 1 2 3 True: Тест нижней границы пройден
exit

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> Сервер остановлен (PID: 29543)

```

Рис. 11. Граф с количеством вершин, равным нижней границе ОДЗ

```

--- Подтест 3.3: Середина ОДЗ ---
True: Сервер tcp запущен на порту 8892 (PID: 29592)
spawn ./client 127.0.0.1 tcp 8892
[CLIENT][TCP] Подключение к серверу...
[CLIENT][TCP] Соединение установлено

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 2
Введите имя файла: test_mid.txt
[CLIENT] Данные успешно считаны из файла 'test_mid.txt'
[CLIENT][TCP] Отправка данных серверу...
[CLIENT] Ответ сервера: Длина: 19; Путь: 0 2 3 4 5 6

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> True: Тест середины ОДЗ пройден
exit
Сервер остановлен (PID: 29592)

```

Рис. 12. Граф с количеством вершин, взятым из середины ОДЗ


```

--- Подтест 3.4: Верхняя граница (100 вершин) ---
True: Сервер tcp запущен на порту 8893 (PID: 29605)
spawn ./client 127.0.0.1 tcp 8893
[CLIENT][TCP] Подключение к серверу...
[CLIENT][TCP] Соединение установлено

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 2
Введите имя файла: test_upper.txt
[CLIENT] Данные успешно считаны из файла 'test_upper.txt'
[CLIENT][TCP] Отправка данных серверу...
[CLIENT] Ответ сервера: Длина: 91; Путь: 0 9 10 11 12 13 14 15 16 17 1
65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 8
8

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> True: Тест верхней границы пройден
exit
Сервер остановлен (PID: 29605)

```

Рис. 13. Граф с количеством вершин, равным верхней границе ОДЗ

```

--- Подтест 3.5: Верхняя граница + 1 ---
True: Сервер tcp запущен на порту 8893 (PID: 29618)
spawn ./client 127.0.0.1 tcp 8893
[CLIENT][TCP] Подключение к серверу...
[CLIENT][TCP] Соединение установлено

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> 2
Введите имя файла: test_upper_UP.txt
[CLIENT] Данные успешно считаны из файла 'test_upper_UP.txt'
[CLIENT] Ошибка: граф должен содержать от 6 до 100 вершин!
Текущее количество вершин: 101
Пожалуйста, введите данные снова.

Выберите источник данных:
1 – ввод с клавиатуры
2 – ввод из файла
Введите 'exit' для выхода
> True: Тест верхней границы +1 пройден
exit
Сервер остановлен (PID: 29618)
Тесты граничных значений завершены

=== ВСЕ ТЕСТЫ ЗАВЕРШЕНЫ ===
Временные файлы удалены

```

Рис. 14. Граф с количеством вершин, на единицу больше верхней границы ОДЗ

На рисунке 15 представлена финальная статистика по тестированию.

```

Статистика
Ожидалось тестов: 10
Выполнено тестов: 10 из 10
Пройдено тестов: 10 из 10
Провалено тестов: 0 из 10

```

Рис. 15. Финальная статистика